inputs to RTL designs. You can achieve this by preparing data files alongside golden models. Alternatively, consider the direct integration of golden models into the Verilog testbench using the DPI. This approach offers the advantage of seamlessly incorporating a wide range of programming languages into the Verilog testbench, thereby boosting flexibility and versatility in your verification process.

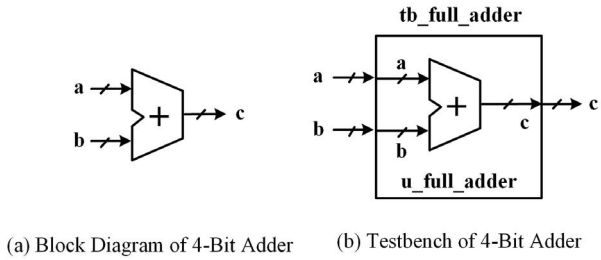
- **Printed Log and Dump Waveform:** Ensure that error messages, along with related signals and precise simulation timestamps, are logged to a designated log file. Furthermore, when conducting random tests, consider dumping waveform data and specifying a unique simulation seed ID. This seed ID facilitates the reproduction of the same random data in subsequent simulations, significantly assisting in efforts related to reproducibility and debugging.
- **Signal Initialization:** Ensure that all design inputs are properly initialized, either with zeros or ones, to avoid unknown signals in the simulation waveform.
- **Regression Testing:** After making changes to the RTL code, it's essential to conduct regression tests to identify potential impacts on other design features. Regression testing should encompass comprehensive verification activities, including coverage analysis, to ensure thorough validation of the design-under-test.

T&F--Not

PBL 6: Four-Bit Full-Adder Design and Simulation

1) Design a 4-bit full adder using the Verilog arithmetic operator "+". Figure 5.8(a) shows the design block diagram, including the 4-bit inputs "a" and "b" and the 5-bit output "c".
2) Create a testbench to simulate the design, encompassing all input combinations of "a" and "b", totaling 256 possible combinations. The testbench shown in Figure 5.8(b) connects the 4-bit signals "a" and "b" to the design inputs, and the 5-bit signal "c" to the design output to monitor the results.

Hint: Repeated loops can be used within the bus functional model to feed in data and within the monitor to automate result validation.



(a) Block Diagram of 4-Bit Adder (b) Testbench of 4-Bit Adder

FIGURE 5.8
Design and simulation specification of 4-bit adder.

PBL 7: Combinational Circuit Design and Simulation

1) Design a combinational circuit named "com_and_or" with an AND gate and two OR gates as shown in Figure 5.9. The design should have single-bit input and output.
2) Create a testbench to simulate the design, encompassing all input combinations of "a-d", totaling 16 combinations.

PBL 8: Sequential Circuit Design and Simulation

1) Using the project presented in Figure 5.9 as a submodule, design a sequential circuit that comprises an instantiation of this submodule along with a register, as depicted in Figure 5.10. The name of the design module is "seq_and_or", and all the IOs are single-bit signals. The register can be triggered by falling reset edges and rising clock edges.
2) Create a testbench to simulate the design, encompassing all input combinations of "a-d", totaling 16 combinations. To simulate the

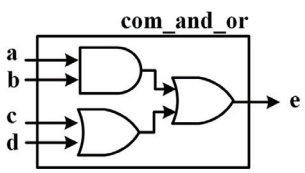


FIGURE 5.9
Combinational AND-OR circuit.

T&F--Not for Distribution

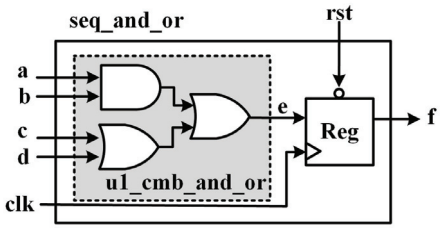


FIGURE 5.10
Sequential AND-OR circuit.

register's behavior, it's essential to have a clock and reset generator in place.

E--Not for Distribution

PBL 9: Sequential Circuit Instantiation and Simulation

1) Using the project in Figure 5.9 as a submodule, design a sequential circuit named "seq_inst_and_or". The sequential circuit should include two submodule instantiations (each consisting of a combinational AND-OR circuit), a single-bit adder, and three registers as depicted in Figure 5.11. All the IOs are single-bit signals. The register can be triggered by falling reset edges and rising clock edges.
2) Create a testbench to simulate the design, encompassing all input combinations of "a-g", totaling 128 combinations. To simulate the register's behavior, it's essential to have a clock and reset generator in place.

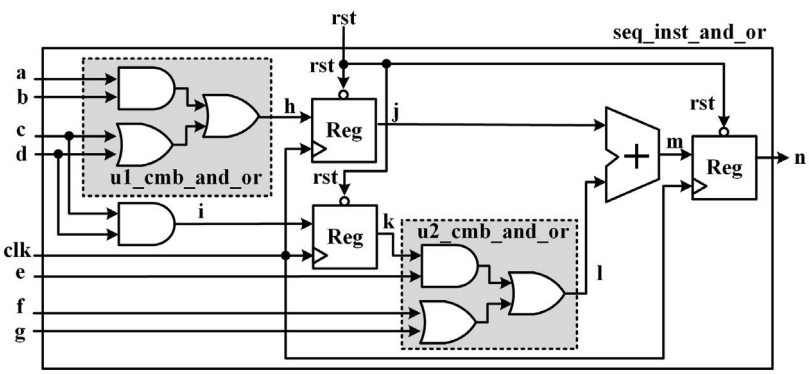


FIGURE 5.11
Sequential circuit instantiation.